

Vision-Based Lane Inference for Unstructured and Structured Indian Roads

Harshwardhan Mukund Mohadikar (2023EBCS353) and Shreyas Bhat K (2023EBCS460) · Supervised by Dr. Ashok Yemineni · BITS Pilani, BSc Computer Science

165/204

FRAMES SOLVED

0.31 m

MEDIAN EDGE ERROR

0.9403

DRIVABLE IOU

27 fps

END TO END

The problem

A lane detector finds painted markings and reports where they are. That works on a motorway and fails on a road with no paint. Much of the Indian network has none, or has paint worn past use, yet traffic on those roads still organises into lanes that drivers agree on. The information is in the geometry of the road surface rather than in its markings.

This project infers lane structure from a single forward-facing camera without requiring markings, reports it in metres, and declines to answer when the scene does not support one. That last part matters: a detector that finds nothing looks exactly like one that has failed, so nothing downstream can tell them apart. Ours reports which it is.

Objectives

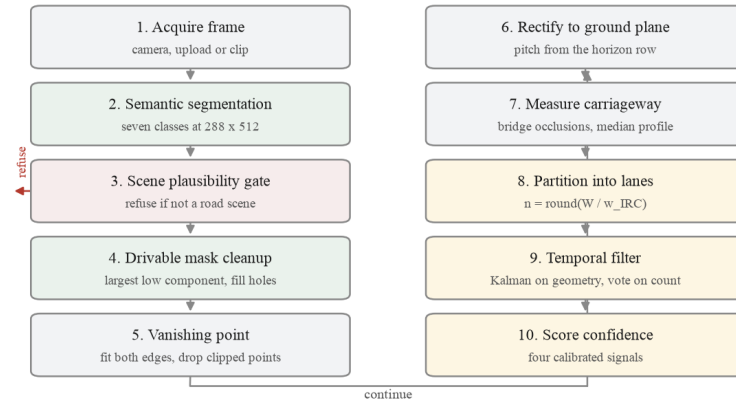
****Primary.**** To review lane detection and inference in the context of Indian road settings; to develop a vision-based lane inference model that works on structured, semi-structured and unstructured roads, recovering lane count and metric width from the geometry of the drivable surface rather than from markings; to make that model optimised, effective and lightweight enough for video-rate operation on commodity hardware; and to validate it against ground truth and against trivial baselines, with a confidence that predicts its own error.

****Secondary.**** To detect wrong-side driving from the inferred lane direction; to examine traffic flow in unorganised road conditions and whether observed traffic can itself indicate lane structure; and to assess the feasibility of extending the system towards basic driver assistance. All three were attempted and are reported with their outcomes, including the one that was measured and rejected.

How it works

Five stages, one forward pass. A MobileNetV3-Large network with an LR-ASPP head and an FPN decoder, 3.33 M parameters, segments the drivable surface into seven classes. The vanishing point of the two road boundaries fixes the camera pitch on every frame, with no calibration target. The ground plane is rectified to a metric bird's-eye view, where a metre is a metre at six metres and at twenty-two. The measured carriageway is divided by the design lane width to give a count. A Kalman filter over the centreline and width, with a majority vote on the discrete count, stabilises the model across video.

Every stage writes an intermediate image, which is what the stage walkthrough in the demo displays.



The pipeline end to end. Each stage is independently switchable, which is what made the ablation possible.

The result that made it work

Recovering metres from one camera normally needs its focal length, and for arbitrary dashcam footage nobody has it. Working the projection through algebraically, it cancels. With pitch derived from the observed vanishing point, lateral distance is

$$dX = du (f H / (v - v_h)) / f = du H / (v - v_h)$$

The focal length appears in the numerator and the denominator and divides out. Measured rather than asserted: sweeping the assumed field of view from 40 to 140 degrees moves the recovered lateral scale by 1.0 per cent, against the 11.8 per cent median width error the system actually exhibits, so it is not the limiting term. That leaves exactly one metric assumption in the whole chain, the camera height, adopted as 1.75 m and swept over five values so a reader can see which conclusions survive a different one.

Results

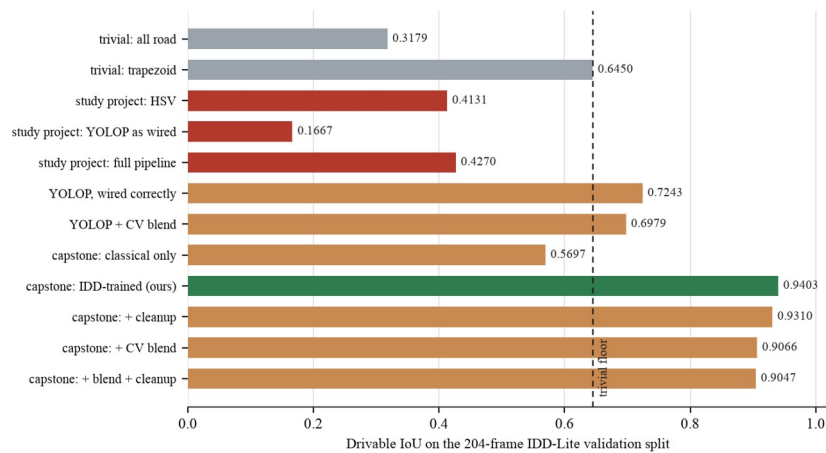
Measured on the IDD-Lite validation split, 204 frames, by an evaluation harness that ships with the code. Every figure here can be regenerated with one command; none is typed in by hand.

CONFIGURATION	DRIVABLE IOU	BOUNDARY F1
Trivial: fixed trapezoid, ignores the image	0.6450	0.1957
Classical computer vision, our first two versions	0.5697	0.2099
YOLOP, as first wired	0.1667	0.1382
YOLOP, preprocessing corrected	0.7243	0.2205
Our network, trained on IDD	0.9403	0.8343
Our network as deployed, with mask cleanup	0.9310	0.7754

We report the deployed number as well as the ceiling. Cleanup costs 0.0093 IoU and the geometry needs the connected mask it produces, so quoting only the higher figure would describe a configuration we do not ship.

Lane geometry against drivable ground truth, over 2,288 boundary samples on 159 comparable frames:

QUANTITY	MEDIAN	MEAN	P90
Road-edge error	0.314 m	0.639 m	1.526 m
Centreline error	0.292 m	0.544 m	1.163 m
Carriageway width, relative	11.8%	22.4%	46.5%



Drivable-area accuracy across twelve configurations. The two grey bars are trivial baselines that never look at the image; a result means nothing without a floor under it.

What it refuses, and why that is the design

The system produces a solution on 165 of 204 validation frames and declines on 39: 20 where the carriageway measures below the 2.5 m plausibility floor, 11 where the road edges are not visible, and 8 where no vanishing point is recovered. Coverage could be made to look better by guessing on those frames. For anything downstream, a confident wrong lane model is worse than none, so it does not.

The reported confidence is calibrated against measured error rather than against how certain the network feels, so gating on it works: at a threshold of 0.8 the median error falls from 0.425 m to 0.339 m. A separate scene check refuses frames that are not road scenes at all, because the network alone cannot abstain: it labels a blank black frame 26 per cent drivable.

The defect that taught us the most

Vehicles standing on the road cut the drivable mask into pieces, so we bridged those gaps before measuring the width. That is right when the gap is a vehicle and wrong when the occluder spans a central median, where bridging joins our carriageway to the opposing one and the measured road doubles. We did not notice for weeks, because we were checking the output against ground truth rather than against the mask it was built from.

Comparing those two settled it immediately: the predicted mask was within 0.20 m of the truth, while the lane model built from that same mask came out 0.58 m wider. The defect was ours, in the geometry, not the network's. Handling occlusion per rectified row instead of per frame took the relative width error at the ninetieth percentile from 151 per cent to 47 and halved the mean edge error. It also made the system refuse six more frames, which we kept: eight of the ten it stopped answering had been in the worst fifth by error.

Stability across video

Measured over 150 clips and 4,526 frames of real video unseen during training. Every frame is measured from scratch, so without smoothing the estimate jitters.

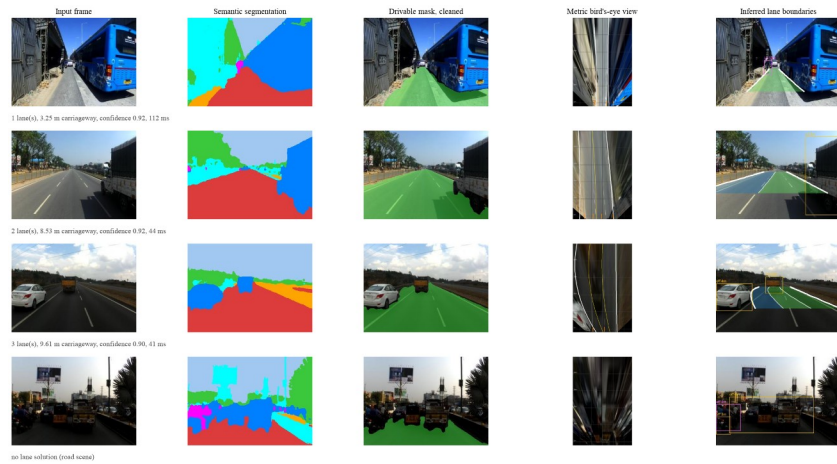
MEASURE	PER FRAME	WITH KALMAN AND VOTE
Carriageway width jitter	0.311 m	0.038 m
Lane-count flips per 100 frames	20.14	4.28
Solution rate	80.4%	85.4%

The last row is the control. A filter that had simply stopped tracking its input would improve jitter while coverage fell; coverage rises, so it is smoothing rather than ignoring.

What was built

A FastAPI backend and a React interface, delivered as a running application rather than a set of scripts. Two routes: a technical report that reads its numbers live from the evaluation output, and a demonstration with six tabs, from a single photograph through to a comparison against the pipeline this replaces.

The deployed image serves the model through ONNX Runtime rather than PyTorch. Only the forward pass ever needed torch and everything after it is numpy and OpenCV, so exporting the network takes the resident set from 564 MB to 226 MB and a CPU forward from 142 ms to 17 ms, with outputs identical across the whole validation split. That is what lets it run on a free 512 MB host.



Four validation frames through the pipeline: a narrow single-lane road, a two-lane road, a painted three-lane road, and a frame the system declines because dense traffic hides both road edges.

What we would tell an examiner before they ask

- Lane counts are inferred from a design standard, not detected, and no Indian dataset carries annotations to validate them. What is validated is the carriageway extent, the 0.314 m figure.
- Metric output rests on an assumed camera height of 1.75 m. Relative error does not move with it; absolute metres scale linearly.
- Ego lane and road-user distances are reported but never evaluated, because IDD has no ground truth for either.
- Trained on daylight imagery. Accuracy falls on wide-angle sensors in low light; four remedies were implemented and rejected on measurement.
- Half the annotated data is unused: IDD-20k Part II was never merged. The preparation script is fixed and the work is ready.
- Not validated for vehicle control, driver assistance, or any safety function.

Deliverables

ARTEFACT	DETAIL
Capstone report	62 pages, with a 32-page concise edition
This summary	5 pages
Presentation	24 slides
Demonstration video	3 minutes 48 seconds, recorded from the running application by a script in the repository
Source code	165 of 204 frames solved by the shipped pipeline; 58 tests, all passing
Measured results	Ten analyses under outputs/results/, read live by the application
Repository	github.com/Arkanobot/VisionBasedLaneInference